

Kriterien für den Nutzen von Root Motion; eine system- und erlebnisbezogene Analyse

Lukas Müller

RWU Ravensburg-Weingarten University, Weingarten.
Deutschland

Die Abgabe der Bachelorarbeit erfolgte am 27. März 2026,
im Studienfach Mediendesign, an der RWU Hochschule
Ravensburg-Weingarten University of Applied Science.

Erstgutachter: Prof. Dr. Daniel Scherzer
Zweitgutachter: Prof. Markus Lauterbach

Angestrebter Abschluss: Bachelor of Science
ETCS: 15 Credits

Abstrakt

In Third-Person Action Games kann die Bewegungssteuerung ein zentraler Interaktionskanal sein. Diese Arbeit untersucht Root Motion als Bewegungssteuerung mit Fokus auf zeitlich begrenzte Charakteraktionen (One-Shot Aktionen). Methodisch handelt es sich um eine literaturgestützte konzeptionelle Analyse, bei der die Kopplung zwischen Bewegung und Pose, bei Root Motion, über drei Kopplungsdimensionen (Quellen-, Zeit-, Blendkopplung) operationalisiert wird. Darauf aufbauend wird zum einen Root Motion architektonisch im Sinne des Separation of Concern Prinzip eingeordnet und zum anderen mit den empirischen Befunden zu Responsivität, Bewegungsplausibilität und Konsistenz der Bewegungssteuerung verknüpft. Die Ergebnisse zeigen, dass Root Motion als Entscheidung über die Bewegungsautorität zu verstehen ist, die zudem die Bewegungsplausibilität durch gestaltbare Sequenzen (über Trajektorie/Timing/Pose) begünstigt. Jedoch können auch Risiken entstehen, durch eine Abhängigkeit innerhalb der Systemschichten oder einer niedrigen oder variierenden Responsivität, die das Kontrollgefühl der Bewegungssteuerung beeinträchtigen können. Aus der Analyse wurden dann Kriterien abgeleitet, die den Nutzen von Root Motion bei One-Shot Aktionen in Third-Person Action Games als plausibel erklären. Root Motion zeigt sich insbesondere dann geeignet, wenn die vorhandenen Animationsclips die gewünschten Bewegungsvarianten abdecken können und eine erhöhte Bewegungsplausibilität sowie das Commitment an Animationssequenzen ein gewünschtes Ziel sind. Ebenfalls soll die Bewegungsautorität explizit im System definiert, die Skalierung des Moveset beherrschbar sein sowie die Bewegungswirkung responsiv und durch Konsistenz erlernbar gehalten werden können.

Keywords: Videospiele, Root Motion, Separation of Concerns (SoC), Bewegungssteuerung, Charakteraktionen, Animationsclip,

1. Einleitung

In vielen Action-orientierten Spielen mit Third-Person Perspektive ist die Bewegung eines spielbaren Charakters der primäre Interaktionskanal. Sie ist meistens das Erste, was eine spielende Person wahrnimmt, und sie bleibt über die gesamte Spielzeit hinweg konstant präsent. Damit bildet die Bewegungssteuerung eine Grundlage des Game-Feel, weil sie bestimmt, wie Eingaben in sichtbares und kontrollierbares Verhalten übersetzt werden. Bereits kleine Verzögerungen, unklare Reaktionen oder abweichendes visuelles Feedback können die wahrgenommene Kontrolle reduzieren und das Erleben einer Aufgabe verändern, vor allem in anspruchsvollen Spielsituationen (vgl. Jörg et. al. 2012, S.1). Root Motion ist eine Technik zum Bewegen von Objekten und Charakteren (Entitäten) in der Spieleentwicklung. In der Praxis wird Root Motion in Third-Person Action Games insbesondere für zeitlich begrenzte Aktionen mit klarer Bewegungssequenz (One-Shot Aktionen wie z.B. Angreifen, Ausweichen) eingesetzt (vgl. Epic Games, 2025a). Weil die Bewegungssteuerung einer großen Verantwortung unterliegt, ist diese Arbeit mit der Motivation entstanden, Root Motion und seinen Einfluss auf das System und Spielerlebnis zu analysieren.

Verschiedene Videospiel-Genres können unterschiedliche Anforderungen an das Gameplay und Steuerung haben, deshalb ist die Arbeit auf Third-Person Action Games beschränkt. Ebenfalls wird Root Motion nicht in Bezug zu speziell einer Game-Engine genommen, da Game-Engines in ihren Features und Methoden variieren können. Dennoch werden Dokumentationen bekannter

Game-Engines für das allgemeine Verständnis zu Root Motion aus der Industrie genutzt, da sich abseits dieser Dokumentationen keine etablierte Standarddefinition außerhalb der jeweiligen Engine-Dokumentationen finden konnte.

Die Arbeit nimmt die Annahme zum Ausgangspunkt, dass die Bewegungssteuerung bei Root Motion die Bewegungsautorität über die Systeme vermischt und in einem Spannungsfeld zwischen Kontrollgefühl und einer realistischen Bewegungsausführung steht. Zudem zeigen Online Foren, dass, obwohl Root Motion eine von den Game-Engines intendiert zu nutzende Technik ist, nicht automatisch klar ist, was die richtigen Voraussetzungen und Anwendungsfälle von Root Motion sind und was für möglicherweise unvorhergesehene Auswirkungen der Nutzen mit sich bringt (vgl. Dori_Yoon, 2023; Pushe, 2020; Vaei, 2016).

Ziel dieser Arbeit ist eine konzeptionelle Analyse, die

- I) durch eine formale Auslegung der Technik die charakteristischen Eigenschaften (Kopplungsdimensionen) von Root Motion operationalisiert und
- II) analysiert, ob die Auswirkungen von Root Motion mit System- und Spielerlebnis-Anforderungen vereinbar sind.
- III) Die empirischen Befunde sollen dadurch eine Grundlage schaffen, aus dem Kriterien abgeleitet werden können, die erklären unter welchen Bedingungen Root Motion hinsichtlich der analysierten Aspekte geeignet ist und die Wahl der Bewegungstechnik in der Entwicklung einer Bewegungssteuerung nachvollziehbar machen.

2. Definition von Root Motion und Operationalisierung der Kopplung

Unter Root Motion wird eine animationsgetriebene Technik verstanden, bei der die Position und Rotation eines Charakters (Entität) aus der Root-Transformation eines Animationsclips abgeleitet und als Bewegungsanteil genutzt wird (vgl. Epic Games, 2025a; O3DE, 2025; Unigine, 2026; Unity, 2025). Praktisch bedeutet dies, der Animationsclip enthält ideal nicht nur eine Pose, als Transformationen der einzelnen Knochen/Gelenke eines Charakters, sondern auch die Wurzeltransformation (Root-Transformation) als Transformation eines zusätzlichen Knochens, den sogenannten „Root-Bone“ (vgl. Epic Games, 2025a; O3DE, 2025). Der Root-Bone ist der erste Knochen in der Hierarchie des Skeletts eines Charakters, dessen Transformation (Translation und Rotation) für jeden Zeitpunkt des Animationsclips definiert ist (Abb.1) (ebd.).

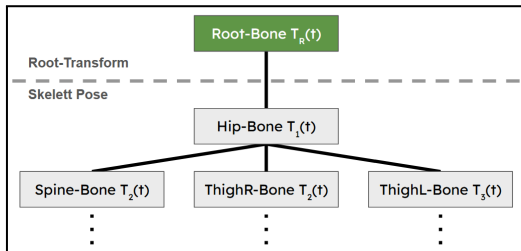


Abb.1: Visualisierung einer möglichen Transformations-Hierarchie eines Skeletts in einem Animationsclip. Die Root-Transformation wird als zusätzlicher Knochen „Root-Bone“ dargestellt und ist der erste Wert in der Hierarchie, der weitere Transformationen des Skeletts beeinflusst.

Zur Laufzeit wird daraus pro Frame die Änderung (Root-Delta) der Root-Transformation bestimmt und als Root Motion Trajektorie interpretiert bzw. auf den Charakter (Entität) angewandt (vgl. Epic Games, 2025a; O3DE, 2025; Unigine, 2026; Unity, 2025).

Um die These aus der Einleitung überprüfen zu können, müssen die Eigenschaften der Technik, also gezielt die Einschränkungen und Abhängigkeiten, die bei Root Motion entstehen, formal herausgearbeitet und veranschaulicht werden. Diese später genannten Kopplungsdimensionen können dann genutzt werden, um dessen Auswirkung auf das System und Spielerlebnis zu analysieren. Diese formale Auslegung von Root Motion geschieht mit zum Teil aus Implikationen, anhand der Beschreibungen innerhalb der Dokumentationen, wie die Schritte und Berechnungen der Technik aussehen können, ohne diesen zu widersprechen.

2.1 Anwendung von Root Motion Formalisiert

Notation und Konventionen.

- Transformationen werden als homogene (4×4) -Matrizen dargestellt. Es wird mit Spaltenvektoren gerechnet (linke Multiplikation)
- t bezeichnet den Abtastzeitpunkt im Clip, Δt die Zeit zwischen dem Frame des Abtastzeitpunkts und dem zu vergleichenden Frame davor.
- Eine Entität hat eine globale Transformation in der Spielwelt $X(t)$ (Welttransformation), Mesh-Vertices $V \in \mathbb{R}^{m \times 3}$ werden als Teil der lokalen Transformation der Entität betrachtet.

Auswertung des Clip-Zustands

Ein Animationsclip liefert zum Zeitpunkt t konzeptionell zwei Anteile: (a) ein globaler Bewegungsanteil durch die Root-Transformation $T_{root}(t)$ und (b) eine Pose

$A_{pose}(t) = (T_1(t), \dots, T_N(t))$, die konzeptionell die Root-neutralen Transformationen aller Knochen repräsentiert (vgl. Epic Games, 2025a; O3DE, 2025; Unity, 2025). Der volle Clip-Zustand ergibt sich als Tupel:

$$A_{full}(t) = (T_{root}(t), A_{pose}(t)) \quad (F1)$$

Root Motion Extraktion und Integration

Root Motion entsteht als per-frame Änderung der Root-Transformation:

$$\Delta T_{root}(t) = T_{root}(t)T_{root}(t - \Delta t)^{-1} \quad (F2)$$

Dieses Root-Delta $\Delta T_{root}(t)$ beschreibt die vom Clip intendierte Bewegung (Trajektorie) einer Entität im betrachteten Zeitschritt (vgl. O3DE, 2025; Unigine, 2026). Die tatsächlich bewegte Entität hat eine Welttransformation $X(t)$. Root Motion wird durch die Integration des Root-Deltas auf diese realisiert:

$$X(t) = \Delta T_{root}(t)X(t - \Delta t) \quad (F3)$$

Damit ist die Entitäts-Transformation über die Zeit die Akkumulation der extrahierten Root-Deltas.

Die Pose $A_{pose}(t)$ wird ohne globale Root-Transformation ausgewertet und erst anschließend über die Entität Transformation $X(t)$ in die Welttransformation überführt (vgl. Unity, 2026). Die Welttransformation der Mesh-Vertices des Charakter ergibt sich als Abbildung:

$$V_{world}(t) = X(t)Skin(A_{pose}(t), V) \quad (F4)$$

(Die Skin Formel stellt das Skinning von Mesh-Vertices an die Bindpose vereinfacht dar, welche zur Berechnung dessen Vertices Positionen verwendet wird. Diese spielt für den weiteren Verlauf keine Rolle.)

Das V bezeichnet die Mesh-Vertices eines 3D Charakter Modells in der Bindpose, der Referenz-Pose als Ausgangszustand, weicht eine Pose von der Bindpose ab, wird diese Abweichung auf die Mesh-Vertices übertragen (vgl. Unity, 2026).

Root Motion wird somit, durch die Zerlegung eines einheitlichen Clip-Zustands in die Veränderung der globalen Entitäts-Transformation $\Delta T_{root}(t)$ und eine lokale Pose $A_{pose}(t)$, verwirklicht. Das Root-Delta wird auf die globale Transformation einer Entität angewendet und die Pose wird auf dessen Mesh-Vertices angewendet.

Blend mehrerer Animationen

Im Einzelclip-Fall werden Root-Transformation und Pose synchron aus demselben Abtastzeitpunkt t gewonnen.

Bei einem Blend entsteht die finale Animation dagegen aus dem Kombinieren (Blending) mehrerer Quellen i (z. B. bei einer Transition von zwei Animationsclips) mit zeitabhängigen Gewichten $w_i(t_i(t))$ und ggf. individuellen Quellzeiten $t_i(t)$ (vgl. Epic Games, 2025a; Unigine, 2026).

$$A_{full,blend}(t) = \text{Blend}(\Delta A_{full}^{(i)}(t_i(t)), w_i(t_i(t))) \quad (F5)$$

Das nun erzeugte $A_{full,blend}(t)$ ist keine neue Clip-Quelle, sondern ein temporärer Animations-Zustand eines Zeitpunkts t , wie in (F1) liefert dieser weiterhin nach der Zerlegung eine Root-Transformation $T_{root,blend}(t)$ und Pose $A_{pose,blend}(t)$.

Diese können wie in (F2) - (F4) genutzt werden. Damit liegt zu jedem Zeitpunkt t trotz Blend ein wohldefiniertes Root-Delta vor, das für die Bewegung der Entität genutzt werden kann.

2.2 Operationalisierung der Kopplung bei Root Motion

Aus der formalen Beschreibung ergeben sich drei miteinander verknüpfte Kopplungsdimensionen, die sowohl die Transformation der Entität als auch die Pose betreffen.

Quellenkopplung

In (F1) zeigt sich, dass Root-Transformation und Pose eines Charakters aus derselben Quelle stammen. Somit sind Bewegung und Pose primär an dieselbe Clip-Quelle $A_{full}(t)$ gebunden und damit nicht frei unabhängig voneinander variierbar. Ein Wechsel des Animationsclip oder der beitragenden Quellen bei einem Blend, beeinflusst gleichzeitig die Veränderung der Welttransformation $\Delta T_{root}(t)$ und die sichtbare Pose $A_{pose}(t)$ einer Entität.

Nach der Berechnung des Root-Delta besteht jedoch die Möglichkeit, Korrekturen an der Trajektorie, unabhängig von der ausgewerteten Pose, vorzunehmen. Z. B. kann das $\Delta T_{root}(t)$ skaliert oder mit einem beliebigen Vektor verrechnet werden, doch dieser Aspekt von "Post-Correction" wird in dieser Arbeit außer Acht gelassen.

Dies ist begründet, denn die Charakterbewegung würde nach solch einer Korrektur immer noch Anteile des Root-Delta beinhalten oder auf diesem aufbauen. Damit ändert sich nichts am Bestand des Einflusses aus dem Quellclip auf die Bewegung. Das $\Delta T_{root}(t)$ wäre in diesem Fall zwar nicht mehr die Veränderung der Welttransformation (Bewegung) selbst, sondern ein Anteil für die endgültige Bewegung. Wenn jedoch die Root-Transformation nach der Auswertung gezielt ausgeglichen und gegebenenfalls ersetzt wird, dann lässt sich argumentieren, dass die Beschreibung von Root Motion nicht mehr zutrifft.

Zeitkopplung

Die zeitliche Auswertung bindet Root-Transformation und Pose an denselben Abtastzeitpunkt t : $\Delta T_{root}(t)$ und $A_{pose}(t)$ werden synchron aus dem Clip abgeleitet.

Die Veränderung der Welttransformation der Entität $\Delta T_{root}(t)$ ergibt sich aus der zeitlichen Differenz Δt der Root-Transformation (siehe (F2)), während die Pose absolut zum Zeitpunkt t interpretiert wird. Damit sind sie über dieselbe Clipzeit miteinander gekoppelt. Änderungen der Abspielgeschwindigkeit oder des Timings wirken sich gleichzeitig auf die Welttransformation der Entität und der Pose aus.

Blendkopplung

In Fällen, in denen Animationen kombiniert werden, wie z.B. in Übergängen oder das Überlagern von Animationsclips, entsteht ein temporärer Animations-Zustand aus der Kombination gewichteter Animationsclips. Für jede beitragende Quelle i , zu einer Quellzeiten $t_i(t)$, wird bei (F5) eine neue Root-Transformation und Pose innerhalb $A_{full,blend}(t)$ gezeugt, die durch dieselbe Gewichte $w_i(t_i(t))$ beeinflusst sind. Damit ist der Blendpfad gemeinsam für das Root-Delta und Pose definiert, nicht nur welche Clips beteiligt sind, sondern wie sie durch $w_i(t_i(t))$ gewichtet werden.

2.3 Root Motion autorisiert Bewegung durch Assets

Eine grundlegende Eigenschaft von Root Motion ist, dass der Nutzen dieser Technik die Veränderung der Welttransformation einer Entität primär zu einem asset-autorisierten Signal macht (vgl. Unigine, 2026). Die per Frame angewandte Transformation entsteht (entweder ganz oder teils) aus einer im Animationsclip enthaltener Root-Transformation (bzw. dessen Delta), die extrahiert und auf die Entität übertragen wird. Damit ist die Bewegung zunächst eine

Trajektorie, die durch Clipwahl und Clipinhalt vorgegeben ist, Variation der Bewegungswirkung entsteht folglich überwiegend über Quellenwahl, Zeit-/Blendlogik, statt über mögliche dynamisch berechnete Bewegungsparameter (wie z.B. Richtung, Zielgeschwindigkeit und Beschleunigung), die für die Berechnung der Trajektorie ohne Animationsclip benötigt werden können (vgl. Anguelov 2019, S.103).

Diese Lage der Bewegungsautorität ist für die weitere Analyse zentral, weil sie den Variationsraum von Bewegungsaktionen praktisch an Clipabdeckung bindet: Werden neue Bewegungsvarianten benötigt (Trajektorie, Geschwindigkeit, Timing), ist der naheliegende Weg häufig die Einführung zusätzlicher Quellen und Übergänge oder womöglich anderer zusätzlicher Steuerlogik, genau die Grundlage für die nachfolgende Betrachtung von Kopplung, Skalierung und Wartbarkeit in Kapitel 3.

2.4 Fazit Root Motion als Bewegungssteuerung

Im Falle von Root Motion ist die Veränderung der Welttransformation $\Delta T_{root}(t)$ einer Entität und die ausgewertete Pose $A_{pose}(t)$ gekoppelt, da sie aus demselben Asset, einem Animationsclip, zu einem gemeinsamen Zeitpunkt t hervorgehen und gegebenenfalls durch ein geteiltes Gewicht $w_i(t)$ kombiniert werden. Bewegung und Pose sind damit strukturell gebunden und können nicht unabhängig variiert werden. Nun kann im Kapitel 3 und 4 II) der Einfluss der Kopplungsdimensionen (Quell-, Zeit-, Blendkopplung) auf das System und Spielerlebnis analysiert werden.

3. "Separation of Concerns" und warum Animationsclip als Medium eine Rolle spielt

Die im Kapitel zuvor definierte Kopplung zwischen Bewegung und Pose hilft zu definieren, wie der Nutzen von Root Motion Einfluss auf die Entwicklung der Systeme für die Bewegungssteuerung nehmen kann. Damit soll die Annahme aus der Einleitung, dass Root Motion die Autorität der Bewegung unter Systeme vermischt, untersucht werden. Um die Auswirkung eines solchen Einflusses messen zu können, wird ein Analyseinstrument benötigt, aus dessen Sicht ein System analysiert werden kann.

Es wurde definiert, dass die Quellenkopplung die Ursache der Bewegung $\Delta T_{root}(t)$ und die der Pose $A_{pose}(t)$ aus derselben Quelle/Animationsclip $A_{full}(t)$ zieht. In diesem Kapitel wird die Folge geschildert, dass die Daten eines Animationsclips im Falle von Root Motion in mindestens zwei Funktionsbereichen genutzt werden: der Simulation des Spielzustands (Gamestate-Schicht) und der Animation des Charakters (Animations-Schicht). Aufgrund dessen wurde das Separation of Concerns Prinzip als Analyseinstrument gewählt.

3.1 Das Separation of Concerns zur Trennung der Gamestate- und Animations-Schicht

Konzeptionell gibt es in interaktiven Systemen (hier Videospielen) einen guten Grund, Gamestate-Schicht und die Animations-Schicht als getrennte Belange zu behandeln. In komplexen Softwaresystemen stehen mehrere Anforderungen und Qualitätsziele gleichzeitig im Raum (z. B. Wartbarkeit, Effizienz, Korrektheit), und diese lassen sich kaum sinnvoll bearbeiten, wenn sie permanent vermischt werden. Dijkstra (1974) begründet in „On the Role of Scientific Thought“ die „Separation of Concerns“ (SoC) als zentrale Technik zur Beherrschung solcher Komplexität. Intelligentes Vorgehen besteht darin, einen Aspekt eines Gegenstands jeweils in Isolation und für seine eigene Konsistenz zu untersuchen (z. B. Wartbarkeit getrennt von Effizienz), statt mehrere Aspekte gleichzeitig zu verfolgen (ebd.).

Übertragen auf Videospiele separiert Anguelov (2019, S.106) in „*Separation of Concerns Architecture for AI and Animation*“ die Systemarchitektur so, dass der Gamestate (Spielzustand) (Regeln, Zustandswechsel, Autorität über Entitätsbewegung) und die Animation (visuelle Ausführung, Posen, Blendkombinationen) sich als zwei unterschiedliche Belange mit unterschiedlichen Anforderungen (Aspekten) definieren lassen. In der hier verwendeten Architekturbetrachtung kann die Gamestate-Schicht vereinfacht als Träger der Zustandsautorität modelliert werden, während die Animations-Schicht auf diesen Zustand reagiert und ein Charakter-Mesh animiert (vgl. Abb. 2). Eine Trennung von Gamestate- und Animations-Schicht ist deshalb nicht nur sauberer Stil, sondern eine Architekturtechnik zur Beherrschung der Komplexität verschiedener Belange. Außerdem erweitert Anguelov (2019, S.109) mit der Forderung, dass visuelle Animationskomponenten keine Gameplay-Zustandsänderungen selbst durchführen, sondern „unter“ der Gameplay-Systeme (Gamestate) geschichtet werden, sodass sie diese nicht direkt beeinflussen können.

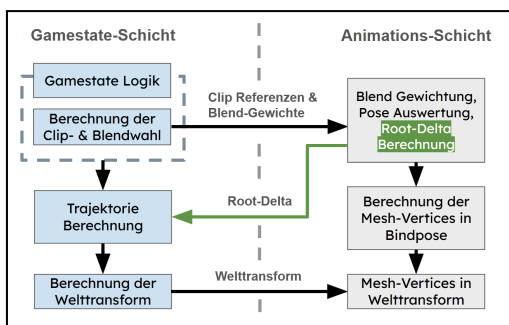


Abb.2: Eine plausible Komposition von Gamestate- und Animations-Schicht im Falle von Root Motion. Durch die Extraktion des Root-Delta stellt Root Motion die Grenzziehung der Systemschichten unter mehr Spannung.

Für Root-Motion-getriebene Charakterbewegung ist diese Grenzziehung nicht trivial, weil mindestens ein Teil der Bewegungswirkung das Root-Delta ist, welches bei der Auswertung eines Animationsclips in der Animations-Schicht berechnet wird (vgl. Abb. 2), und damit nicht ausschließlich in der Gamestate-Schicht liegt. In diesem Fall konsumiert die Animations-Schicht nicht nur Daten aus der Gamestate-Schicht, sondern trägt auch die Verantwortung zur Charakterbewegung. Da jedoch eine Gameplay-Zustandsänderung aus der Animations-Schicht laut dem SoC unerwünscht ist, wird das Root-Delta nur an die Gamestate-Schicht übergeben, die dann über den Gebrauch entscheidet (vgl. Anguelov 2019, S.109). Zwar werden so die Systemschichten getrennt gehalten, jedoch könnte man argumentieren, dass die Grenzziehung nicht ganz klar ist, da nun eine Datenabhängigkeit in beide Richtungen besteht.

3.2 Animation-Driver-State-Machine als Interface der Systemschichten

Anguelov (2019, S.108) zeigt, dass bei der Trennung von Gameplay- und Animations-Belangen (die unabhängig von Root Motion geschieht) eine zusätzliche, zuvor implizite Steuerlogik innerhalb der Gamestate-Schicht sichtbar wird. Diese Schicht ist in diesem Kontext ein Interface, bestehend aus einer parametergesteuerten, zustandsbasierten Animationssteuerung mit Blendstruktur, die die Verantwortung der Clip- und Blendwahl trägt. Diese Verantwortung wird aus der Gamestate-Schicht entnommen und in eine eigene, jetzt dritte Systemschicht transferiert, ein Indiz für zuvor vermischte Verantwortlichkeiten (vgl. Anguelov 2019, S.108). Für den weiteren Verlauf der Arbeit wird diese dritte Schicht als Animation-Driver-State-Machine-Schicht (ADSM-Schicht) bezeichnet, da die „Animation Controller“ Schicht, wie sie von Anguelov bezeichnet wird, noch mehr beinhalten kann als die ADSM, auf die ausschließlich der Fokus in diesem hier liegt.

Eine entscheidende Erkenntnis aus Anguelov's Kapitel (2019, S.105) ist, dass bei dieser neuen Steuerungsschicht (ADSM-Schicht) die Komplexität nicht nur auf „Daten“ beschränkt bleibt. Wenn das ADSM Interface Transitionen triggert, Clip auswählt und gewichtet, entsteht außerdem eine Strukturabhängigkeit. Zum Erfüllen einer bestimmten Animationswirkung muss der treibende Code jedoch auch detailliertes Wissen darüber haben, welche Zustands- und Blend-Parameter in der ADSM existieren, wie sie zu interpretieren sind und wie Werte in die jeweils erwarteten Steuergrößen zu konvertieren sind, um einen gewünschten Zustand zu erreichen (ebd.). Das ist Wissen, das die Gamestate-Schicht nach der Trennung nicht mehr zwingend selbst besitzt. Daher können Änderungen an der ADSM-Struktur (z. B. Blendpfade) das Gameplay invalidieren und Codeänderungen erzwingen (ebd.).

In Abbildung 3 fungiert die ADSM-Schicht als Interface zwischen Gamestate- und Animations-Schicht, bei der die Gamestate-Logik nur noch indirekt über ausgewählte Parameter den finalen Animations-Zustand $A_{full,blend}(t)$ beeinflussen kann. Parameter sind nur an einem Ort definiert, hier in der ADSM-Schicht. Wenn die Gamestate Logik diese Parameter verändert, passiert das über Annahmen, welche Auswirkungen dies hat. Ein praktisches Beispiel wäre die Einführung einer aufladbaren Jump-Aktion, dann muss z.B. zuerst ein Parameter für die Aufladedauer, ein Trigger für den Jump Moment und eine Blendstruktur vom aktuellen Zustand zum Jump-Zustand existieren, damit die intendierte Bewegungsaktion genutzt werden kann. Wenn dieser Jump-Zustand erreicht werden soll, muss die Gamestate-Logik die Parameter kennen und gezielt nutzen können. Die Folge ist, dass neue Charakteraktionen neue Parameter benötigen und womöglich spätere Änderungen erschweren können.

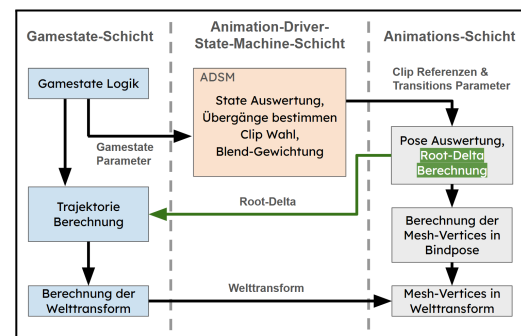


Abb.3: Die Animation-Driver-State-Machine als zusätzliche Steuerschicht. Diese nimmt der Gamestate-Schicht die Verantwortung für Clipwahl und Blend.

3.3 Der Animationsclip als Medium

Genau an dieser Stelle wird der Animationsclip als Medium architektonisch relevant. Bei der zuvor operationalisierten Querkopplung ist der Clip nicht nur ein Artefakt für den Nutzen der Charakterpose, sondern zugleich eine treibende Bewegungsquelle, weil er nicht nur die Pose, sondern auch die Bewegung (Root-Delta) liefert.

Konkret kann das bedeuten, während in Abbildung 2 die Gamestate Logik die zusätzliche Verantwortung für die Berechnung des Root-Deltas hat, über die Kontrolle von Clipwahl $A_{full}(t)$, Zeitpunkt t und möglichen Blendgewichten $w_i(t_i(t))$, ist im Falle einer ADSM-Schicht nur ein indirekter Einfluss aus der Gamestate-Schicht möglich (vgl. Abb. 3). Dadurch kann die Isolierung der Gamestate-Schicht erschwert sein, da die Daten und Prozesse zum Treiben der Bewegung nicht allein in der Gamestate-Schicht liegen, sondern auch in der ADSM-Schicht. Anpassungen an Bewegungsreichweite, Trajektorie oder Varianten einer Aktion werden nicht nur als unabhängige Gameplay-Parameter innerhalb der Gamestate-Schicht modelliert, sondern hängen explizit an Clip- und Blendwahl ab, die in der ADSM-Schicht liegen. Das

verstärkt die von Anguelov (2019, S.104) beschriebene Koordinationskosten zwischen Verantwortlichkeitsschichten. Die Folge ist, dass bei Root Motion die Bewegung einer Entität nicht nur ein Belang der Gamestate-Schicht bleibt, sondern auch explizit der Systemschicht, in der Entscheidungen über die Anwendung von Animationsclips gemacht werden, was genau auf die ADSM-Schicht zutrifft.

Allerdings ergibt sich in der Praxis der Nutzen einer solchen Interface-Schicht, zwischen dem Spielzustand und der Animation eines Charakters, auch als naheliegende Reaktion. Besonders dann, wenn einzelne Systemschichten mehrere Verantwortungen übernehmen. Aus einem anderen Blickwinkel ergibt sich in Abbildung 3 eine System-Schicht (ADSM-Schicht), die gezielt allein die Verantwortung für die Übersetzung des Gamestate-Zustands in eine Animation/Pose trägt und die Animationsclips handhabt. Dort können Root-Transformation und die Charakterpose gemeinsam innerhalb einer ADSM visualisiert und synchron bearbeitet werden. Daraus ergibt sich für die Bewegungssteuerung ein Trade-Off zwischen:

(a) eine Bündelung und das Bearbeiten von Clip- und Blendwahl durch Zustands- und Transitionslogik in einer Systemschicht,

(b) eine neue Strukturabhängigkeit der Gamestate- und ADSM-Schicht über Parameter, die die Clip- und Blendwahl über die ADSM-Struktur steuern.

3.4 Kopplungs- und Skalierungsprobleme

Anguelov (2019, S.106) ordnet Daten- und Strukturabhängigkeiten als Verletzung des Separation of Concerns Prinzips ein. Interagierende Systeme sollen klare, nicht überlappende Verantwortlichkeiten besitzen. Als Kernmechanismus für Skalierungsprobleme beim Treiben der Animationen beschreibt er Code/Daten-Abhängigkeiten mit dem Gamestate-Code (vgl. Anguelov 2019, S.104). Damit die Gameplay Logik die Animationen in Abbildung 2 direkt steuern kann, muss es entweder Clip-Daten, die Zustandsmaschine der Animationen und deren Blendpfade selbst kontrollieren oder bei einer ADSM-Schicht konkretes Wissen darüber besitzen (ebd.). Änderungen an Animationsclips, der Zustandsmaschine oder der Blend-Regelung können Codeanpassungen erzwingen und umgekehrt (ebd.).

Gleichzeitig ergibt sich eine praktische Limitierung des Bewegungsaktionsraums, weil jede zusätzliche Bewegungsvariante typischerweise zusätzliche Clips, ADSM-Zustände, Übergänge und Parameterlogik nach sich zieht, also auch genau jene Skalierungsprobleme, die Anguelov (2019, S.104) als Wartungs-/Debugging-Risiko und als teurer werdende Erweiterbarkeit beschreibt.

Damit ist die SoC-Perspektive in dieser Arbeit kein abstraktes Ideal, sondern ein Analyseinstrument. Wenn der Animationsclip als Medium gleichzeitig Charakteranimation und Bewegungsautorität trägt, erhöht sich die strukturelle Kopplung zwischen Gamestate- und Animations-Schicht, was die Isolierung der Systemschichten erschweren kann.

3.5 Warum die Kopplung trotzdem attraktiv ist

Trotz der ausführlich beschriebenen SoC-Konflikte kann die Kopplung von Bewegung und Pose bei Root Motion in der Praxis attraktiv sein, weil sie Bearbeitungs-Kontrolle (an Trajektorie, Timing und Pose) in einem gemeinsamen Datenartefakt (Animationsclip) bündelt. Die Animationsclips können in einer Systemschicht gehandhabt werden, die speziell für diesen Zweck eingeführt werden kann. Root-Transformation und Pose werden zusammen als Animations-Zustand $A_{full}(t)$ oder Kombination mehrerer Clips $A_{full,blend}(t)$ betrachtet und lassen sich gemeinsam bearbeiten, dies kann die Wahrnehmung einer kohärenten und plausiblen Bewegungsausführung begünstigen, ein Qualitätsmerkmal der Bewegungssteuerung (vgl. Kap.4.1 & 4.3). Zum Beispiel kann sowohl die Pose-Ausführung als auch die korrekte Trajektorie

(Bewegung zur Schrittfolge der Pose) einer Angriffsaktion allein durch die Entscheidung über die Clipwahl innerhalb einer ADSM verwirklicht werden.

Die Bewegungswirkung ist damit Teil der Charakteraktions-Animation selbst. In der Praxis kann dieses von einem Designer gestaltete Zusammenspiel von Bewegung und Pose eines Animationsclips insbesondere für klar abgegrenzte One-Shot Aktionen relevant sein, bei denen komplexe, inszenierte Trajektorien (inkl. Rotationsanteilen und Schrittfolgen) als einzelnes Asset gestaltet werden, anstatt durch ein mögliches separates Bewegungsmodell rekonstruiert und zusammengefügt werden zu müssen. Ein clipbasiertes Bearbeiten kann dann das Feinanpassen von Übergängen und Bewegungssequenzen auf Ebene der Animationsinhalte erleichtern.

Dies wirkt als ein klarer Produktionsvorteil, wenn an der Entwicklung der Bewegungssteuerung im Zusammenspiel mit der visuellen Ausführung Zeit und Arbeit gespart werden kann.

4. Spielerlebnis: Qualitätsmerkmale der Bewegungssteuerung

Die Kopplungsdimensionen von Root Motion aus Kapitel 2.2 können im Spielerleben unmittelbar wirksam werden, daher wird hier die Annahme aus der Einleitung geprüft, ob die Bewegungssteuerung bei Root Motion eine Spannung zwischen Kontrollgefühl und einer realistischen Bewegungsausführung schafft. Demnach werden in diesem Kapitel zuerst Qualitätsmerkmale für die Bewegungssteuerung basierend auf einer Arbeit zu Game-Feel abgeleitet, daraufhin wird der Einfluss der Kopplungsdimensionen auf diese Qualitätsmerkmale ermittelt.

4.1 Qualitätsmerkmale der Bewegungssteuerung als Analysemaßstab

Pichlmair und Johansen (2020, S.1) definieren Game-Feel an sich als intentionalen Entwurf des affektiven Impacts von Moment-zu-Moment-Interaktion und ordnen Gestaltungsmaßnahmen den Domänen Physicality, Amplification und Support zu: (1) Bewegungsreaktionen sollen kohärent, erlernbar und damit vorhersagbar sein (Physicality), (2) Bewegungs- und Feedbacksignale sollen die Bedeutung und Wirkung einer Aktion klar kommunizieren (Amplification), und (3) Das Steuerungssystem soll die intendierte Spielerhandlung zuverlässig in eine ausführbare Aktion überführen und Korrekturen zulassen, sofern das Aktionsdesign sie nicht bewusst einschränkt (Support).

Ergänzend beschreiben praxisnahe, spielerzentrierte Beobachtungen zu Third-Person-Kampfspielen das Gefühl von Kontrolle als Ergebnis vorhersagbarer, unmittelbar rückgekoppelter Reaktionen, visuelle Prinzipien wie Timing sowie Blend-in/Blend-out kommunizieren dabei Gewicht und Tempo, und „instant feedback“ verstärkt das Kontrollgefühl (vgl. Lindgren et. al. 2025, S.14).

Für den weiteren Verlauf dieser Arbeit werden deshalb die Qualitätsmerkmale in dieser Arbeit bewusst so gefasst, dass sie auf die Bewegungssteuerung anwendbar sind, nicht auf die gestalterische Qualität einzelner Animationsclips und zudem mit den Begriffen „Responsiveness“ und „Naturalness“ aus den später aufgegriffenen Arbeiten von Normoyle und Jörg et. al. (2012, 2014a, 2014b) korrelieren:

Responsivität

Das System kann eine intendierte Aktion zum erwarteten Zeitpunkt auslösen und Eingaben ohne unnötige Verzögerung wirksam werden lassen.

Bewegungsplausibilität

Die wahrnehmbare Ausführung wirkt als Bewegungsverlauf plausibel und kohärent (mit der Charakter-Pose), sodass die Aktion verständlich durch die Ausführung getragen wird.

Konsistenz

Die räumliche und zeitliche Ausführung einer Aktion bleibt stabil genug, um erlernbar und vorhersagbar zu sein.

Zudem ist relevant, dass Aktionen häufig als zeitlich strukturierte Sequenzen gestaltet werden (z. B. Antizipation/Impakt/Erholung) (vgl. Lindgren et. al. 2025, S.14). Diese Sequenzierbarkeit ist bei Root Motion durch den zeitgekoppelten Animationsclip gegeben und wird später in Kapitel 4.5 als Gestaltungshebel aufgegriffen.

Vor diesem Hintergrund stellt sich die Frage, inwiefern die zuvor operationalisierten Kopplungsdimensionen (Quellen-, Zeit- und Blendkopplung) diese Qualitätsmerkmale beeinflussen und damit Kontrolle, Lesbarkeit und Timing in Charakter-Aktions-Situationen systemisch mitprägen.

4.2 Responsivität

Empirische Arbeiten fassen Responsivität als Zeitspanne zwischen Eingabe und wahrnehmbarer Reaktion zusammen, kürzere Latenz entspricht höherer Responsivität (Reaktionszeit) (vgl. Jörg et. al. 2012, S.1; Normoyle und Jörg 2014a, S.1). In einem Third-Person-Plattform-Setting zeigen Jörg et. al. (2012, S.5), dass bereits moderate Verzögerungen messbare Auswirkungen auf wahrgenommene Kontrolle und Performanz haben können. Es wurde zudem oft von verringertem Kontrollgefühl und Frustration berichtet, ohne dass die Verzögerung zwingend als Ursache benannt wurde (ebd.). Der Effekt fällt unter höherer Anforderungslast stärker aus, was die Relevanz schneller Responsivität in anspruchsvollen Interaktionssituationen unterstreicht (vgl. Jörg et. al. 2012, S.1).

4.3 Bewegungsplausibilität und der Trade-Off zur Responsivität

Die Responsivität von der Charakterbewegungen ist nicht isoliert zu betrachten, denn sie steht in einem Spannungsfeld (einem Trade-Off) zur Bewegungsplausibilität. Normoyle und Jörg (2014a, S.2) definieren die Bewegungsplausibilität (Naturalness) als wahrgenommene Realistik der Bewegung. In der Arbeit ist die Evidenz, dass die Bewegungsplausibilität durch die glaubhafte Ausführung (Plausibilität) des Aktionsdesigns und durch das kohärente Zusammenspiel von Bewegung und Pose entsteht (vgl. Normoyle und Jörg 2014a).

Es wurden explizit die Verzögerung bei Bewegungswechsel untersucht, die durch Übergänge und Bewegungssteuerung resultieren: (1) entweder wird die Bewegungswirkung sofort umgesetzt (hohe Responsivität durch abrupte Übergänge) oder (2) über geglättete Übergänge mit verzögerter Bewegungswirkung (realistische, plausible Bewegung) (vgl. Normoyle und Jörg 2014a). Ihre Experimente zeigen, dass eine höhere Responsivität im Spielkontext stärker mit Kontrolle und Leistung korreliert als maximale Bewegungsplausibilität (vgl. Normoyle und Jörg 2014a, S.1). Die am wenigsten responsive Bewegungssteuerung verschlechterte Performanz und wahrgenommene Kontrolle, zugleich waren Spielende im Mittel am zufriedensten mit der eigenen Performanz beim am wenigsten „natural“ Controller (statische Pose) (ebd.). Zudem bewerteten Spielende die Bewegungsqualität des „most natural“ Controllers nicht höher, obwohl Außenstehende (nicht Spieltester) ihn als deutlich „natürlicher“ wahrnehmen (ebd.). Die Bewegungsplausibilität dominiert in der Gesamtbewertung nicht automatisch, „mehr Naturalness“ ist kein automatischer Qualitätsgewinn, wenn sie mit Kontrollverlust oder Verzögerung erkaufte wird.

In Kapitel 3.5 konnte festgestellt werden, dass Root Motion das eine plausible Bewegungsausführung durch das kohärente Zusammenspiel von Bewegung und Pose begünstigt. Daher ist der Nutzen dieser Technik besonders bei Bewegungsplausibilität als Zielsetzung geeignet. Jedoch zeigte sich, dass Bewegungsplausibilität nicht automatisch dominiert, daher ist es eine legitime Lösung, die Bewegungsautorität zugunsten von Kontrolle/Timing anders zu verteilen, sprich Root Motion nicht zu nutzen, um Bewegung und Pose getrennt adressieren zu können.

4.4 Aktionskonsistenz

Neben der absoluten Verzögerung ist die Konsistenz der Reaktionszeit ein eigenständiger Faktor von Kontrolle. Normoyle et. al. (2014b, S.1) unterscheiden konstante Latenz von Jitter (variierender Verzögerung) und zeigen, dass konstante Verzögerungen in ihrem Setup die Spielerfahrung nicht zwingend verschlechtern, sofern sie stabil bleiben. Sobald jedoch Jitter hinzukommt, verändert sich die Spielerfahrung deutlich. Jitter wird stärker bemerkt, die Leistung sinkt und insbesondere das Erlernen der Steuerung wird beeinträchtigt, da sich kein vergleichbarer Learning-Effekt wie bei konstanten Verzögerungen ergibt, zusätzlich verschlechtert sich die wahrgenommene Bewegungsqualität (ebd.). Für Charakter-Aktions-Design ist dieser Befund unmittelbar anschlussfähig, weil Steuerungsprobleme nicht nur als langsam, sondern auch als unzuverlässig erlebt werden können.

Diese Variante der Verzögerung kann womöglich auch auf das Kombinieren von Animationsclips bei einem Blend übertragen werden. Bei einem Blend von einer Animation zu einem anderen (z. B. Idle zu Angriff) wird in einem Zeitfenster das Root-Delta aus einem kombinierten Zustand gewonnen (vgl. (F5)). Dieser Zustand zu einem Zeitpunkt t ist abhängig von allen beitragenden Clip-Quellen, wenn die Eintritts-Animation jedoch eine andere ist (z. B. Ausweichen zu Angriff), dann hat das direkten Einfluss auf das berechnete Root-Delta zum Zeitpunkt t , also eine Varianz der Aktionswirkung (vgl. Abb. 4).

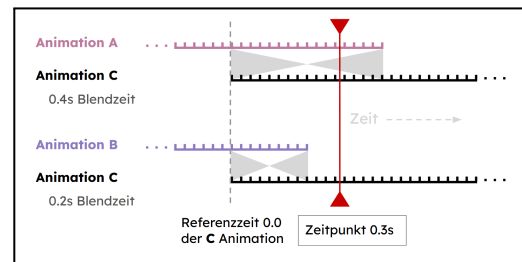


Abb.4: Eine Darstellung der Blendpfad-abhängigen variierenden Bewegungswirkung. Zum Zeitpunkt 0,3s, bei einem Blend von Clip A nach C, ergibt sich die Bewegungswirkung aus einer Kombination beider. Beim Blend von B nach C ist zu dem Zeitpunkt der Blendvorgang schon vorbei, wodurch die Bewegungswirkung nur aus dem Clip C stammt.

Daraus lässt sich als Transferhypothese ableiten, dass auch zustandsabhängige Unterschiede in der Bewegungswirkung, etwa wenn Ausgangszustände oder Übergangslogik einer Aktion deren effektives Timing oder Stärke variieren, die Vorhersagbarkeit und Lernbarkeit von Aktionen reduzieren können. Je größer die Anzahl an verschiedener Ausgangszustände, desto stärker könnte die Lernbarkeit der Steuerung darunter leiden.

4.5 Einfluss der Kopplungsdimensionen auf die Qualitätsmerkmale der Bewegungssteuerung

Vor diesem Hintergrund lassen sich die zuvor operationalisierten Kopplungsdimensionen von Root Motion (Quell-, Zeit- und Blendkopplung) als Mechanismen einordnen, die genau diese Anforderungen der Bewegungssteuerung beeinflussen können.

Quellenkopplung bindet Bewegung und Pose an dieselbe Clip-Quelle, damit hängt die Erreichbarkeit einer gewünschten Bewegungswirkung von der Verfügbarkeit und Kombinierbarkeit passender Clipsegmente ab und kann nur gemeinsam gestaltet werden. Bei einem clipbasierten Controller, in der Trajektorien und Timing aus Clips stammen, kann die mangelnde Animationsclip-Abdeckung ein grundlegendes Risiko solcher Controller benannt werden (vgl. Kap. 2.3). Wenn die gewünschte Variante nicht abgedeckt ist, ist das nicht nur eine „fehlende Animation“, sondern eine Kontroll-/Intentionserfüllungsfrage, (Support-Domäne laut Pichlmair und Johansen (2020, S.1)).

Die Lage der Bewegungsautorität kann einen attraktiven Einfluss auf das Kontrollgefühl haben, weil wenn bei Root Motion die Bewegung und Pose eines Charakters aus demselben Asset stammen, gleich beeinflusst werden und dadurch kohärent gestaltbar sind, kann das die Bewegungsplausibilität begünstigen. Für jede durch den Clipbestand autorisierte räumliche Bewegung gibt es eine passende Pose, die gezielt in ihrem Animationsdesign an sich auf Bewegungsplausibilität abgestimmt werden kann.

Zeitkopplung beschreibt, dass die Bewegungswirkung und Pose denselben Clip-Zeitparameter teilen. Die wahrnehmbare Erfüllung einer Eingabe ist damit an Clipauswertung und Blend/Übergangsdauer gebunden. Normoyle und Jörg (2014a, S.1) machen diesen Punkt als Forschungsfokus explizit, indem sie Verzögerungen durch das Animationssystem untersuchten, außerdem benennen sie die Zeitspanne von Transitionen zwischen Animationsclips als potenzielle Ursache solcher Verzögerungen in realistischen Bewegungsskizzen. Für Charakter-Aktions-Design folgt daraus: Bewegungsplausibilität-steigernde Übergänge können die Reaktionszeit systematisch verlängern, ohne dass dafür eine Netzwerk- oder Input-Verzögerung nötig ist, was als niedrige Responsivität resultiert.

Ebenfalls ist die Zeitkopplung bei Aktionen nicht nur eine potenzielle Quelle von Verzögerung, sondern auch ein Gestaltungshebel. Für One-Shot Aktionen ist es üblich, die Bewegungsautorität temporär an eine zeitlich-begrenzte Sequenz mit klaren Phasen (Aufholen, Impact, Erholen) zu übertragen, sodass Trajektorie und Pose als zusammenhängendes Bewegungsmuster ausgeführt werden können (vgl. Epic Games 2025b; Lindgren et. al. 2025, S.14). Diese Sequenzierbarkeit unterstützt Game-Feel-Ziele wie Gewicht, Lesbarkeit und Verbindlichkeit (Amplification). Playtests zeigen etwa, dass längeres Aufholen bei einem Angriff die Erwartungen einer stärkeren Wirkung erzeugen, während lange Erhol-Phasen als „sluggish“ und weniger kontrolliert erlebt werden (Lindgren et. al. 2025, S.30). Es besteht jedoch die Möglichkeit diese Sequenzen zu unterbrechen, doch ob und wann eine Aktion unterbrechbar ist, bleibt eine Entscheidung der Steuerlogik (Gamestate Logik oder ADSM) (ebd.). Die Zeitkopplung liefert jedoch das besonders geeignete Medium, um Commitment als zeitparametrisierte Sequenz zu verfassen und die Wirksamkeit von Übergängen konsistent an Clip-Sequenzphasen und Übergangsdauern zu binden.

Blendkopplung koppelt die resultierende Bewegungsantwort an denselben Blendpfad wie die Pose. An dieser Stelle wird Bewegungsplausibilität zentral, denn ein Blend ist gerade der Mechanismus, der in Normoyle und Jörg (2014a, S.2) Bewegungsplausibilität repräsentiert (glattere Übergänge), aber eben mit Responsivität im Konflikt. Die Bewegung und Pose aus einem Animations-Zustand (Clip) können nicht unabhängig voneinander verschiedene Ziele verfolgen, z. B. eine schnelle Bewegungswirkung und gleichzeitig ein langsamer Posen-Übergang.

Außerdem kann die effektive Responsivität und Bewegungsplausibilität Zustands- und Blend-abhängig variieren, z. B. abhängig davon, aus welchem Ausgangszustand eine Aktion eingeleitet wird und wie Transitionen parametrisiert sind. Als Transferhypothese lässt sich ableiten, dass zustandsabhängig variierende Übergangsdauern die effektive Reaktionszeit weniger vorhersagbar machen können und damit Lernbarkeit beeinträchtigen könnten. Diese Annahme ist plausibel, da Variabilität der Verzögerung (Jitter) in empirischen Studien sich stärker negativ auf das Kontrollgefühl auswirkt als konstante Verzögerung.

4.6 Fazit

Hier nochmal zusammengefasst, die empirischen Befunde stützen folgende Kernaussagen in Bezug auf die Qualitätsmerkmale für das Spielerlebnis bei Third-Person Action Games:

- (1) Responsivität beeinflusst Kontrolle, Frustration und Leistung, insbesondere unter anspruchsvollen Bedingungen.
- (2) Die Konsistenz der Reaktionszeit ist ein Qualitätsfaktor für die Erlernbarkeit der Steuerung und kann die Leistung stärker

beeinträchtigen als eine niedrige Responsivität.

(3) Bewegungsplausibilität ist im Spielkontext nicht automatisch dominant, sondern konkurriert mit einer hohen Responsivität und kann sogar hinter ihr zurücktreten.

Es könnte gezeigt werden, dass der Nutzen von Root Motion interaktive Bewegungssteuerungen direkt beeinflussen kann, wie die Qualitätsmerkmale erfüllt oder nicht erfüllt werden können. Damit lässt sich Root Motion, über die Kopplungsdimensionen und Verteilung der Bewegungsautorität, besser verstehen. Es kann als Designstrategie verstanden werden, bei der Bewegungsplausibilität über die Kopplungsdimensionen begünstigt werden kann, während Responsivität und dessen Konsistenz in Blend-Übergängen nicht automatisch gegeben und zu stabilisieren sind.

5. Kriterien für den Nutzen von Root Motion

Die Kapitel 2–4 haben Root Motion eine Kopplung zwischen Bewegung und Pose über die Kopplungsdimensionen definiert, die daraus folgenden SoC-Spannungen beschrieben und den Einfluss von Root Motion auf die Qualitätsmerkmale der Bewegungssteuerung analysiert. Kapitel 5 nutzt diese Ergebnisse, um Entscheidungskriterien für den Nutzen von Root Motion abzuleiten. Es wird in diesem Kapitel herausgearbeitet, unter welchen Bedingungen Root Motion in One-Shot Aktionen hinsichtlich Bewegungsautorität im System, Kontrollgefühl und Bewegungswirkung geeignet ist.

5.1 Kopplungsdimensionen als Wirkpfade

Um die Ableitung von Kriterien im nächsten Abschnitt besser verständlich zu machen, werden die drei Wirkpfade, Quell-, Zeit- und Blendkopplung, hier nochmal wiederholt zusammengefasst, mit den Erkenntnissen, welchen Einfluss die jeweiligen Kopplungen auf das System und Steuerungsgefühl haben können:

- **Quellenkopplung:** Root-Delta und Pose stammen aus derselben Clip-Quelle, Bewegungsvarianten sind damit an Clipbestand und Clipwahl gebunden und können nicht unabhängig variiert werden. Die Erfüllung der Bewegungsintention ist durch den Clipbestand limitiert. Doch durch einen Clip decken sich Trajektorie und Pose gegenseitig ab, was die Bewegungsplausibilität verstärken kann. In einer Systemschicht, die speziell für die Handhabung der Animationsclips ist (z. B. ADSM-Schicht), ist die Bewegungsautorität auch ein Belang dieser. Die Verflochtenheit der Systemschichten kann die Komplexität im System erhöhen und der Wartbarkeit schaden. Einfluss auf die Bewegungswirkung hat der Spielzustand nur noch indirekt über Parameter, da die Clip- und Blendwahl von der ADSM abhängig ist.
- **Zeitkopplung:** Root-Delta und Pose teilen denselben Clip-Zeitparameter, Aktionen sind als gemeinsame Sequenzen gestaltbar, aber die Wirksamkeit einer Aktion kann durch Übergänge/Sequenzzeit verzögert werden, was direkten Einfluss auf Responsivität und Bewegungsplausibilität hat.
- **Blendkopplung:** Root-Delta und Pose werden durch dasselbe Blendgewicht je gleich beeinflusst und folgen dadurch denselben Blendpfad. Die Bewegungstrajektorie kann Zustands- und pfadabhängig variieren und damit die Konsistenz und Lernbarkeit der Steuerung beeinflussen.

5.2 Ableitung von Kriterien: Bedingungen der Eignung von Root Motion

Ziel III) aus der Einleitung wird hier umgesetzt: Aus den Kopplungsdimensionen (Kap. 2), den SoC-Folgen (Kap. 3) und den empirisch gestützten Spielerlebnis-Anforderungen (Kap. 4) werden

Entscheidungskriterien abgeleitet. Diese Kriterien sind keine universellen Regeln, sondern Bedingungen, die erklären, wann Root Motion hinsichtlich der Systemwartung nach dem SoC, dem Kontrollgefühl und der Bewegungswirkung plausibel ist und wo Risiken entstehen.

Da Root Motion in dieser Arbeit insbesondere im Kontext zeitlich begrenzter Sequenzen betrachtet wird, liegt der Fokus bei den abgeleiteten Kriterien in diesem Kapitel weiterhin auf den One-Shot Aktionen.

Kriterium 1:

Die Bewegungsautorität ist explizit definiert und stabil.

Root Motion ist dann geeignet, wenn klar festgelegt wird, welche Zuständigkeiten in den jeweiligen Systemschichten verbleiben. Die Bewegungsautorität zeigt sich bei Root Motion assetorientiert, denn es zeigt sich, dass das Asset (Animationsclip) und die Daten nach der Clipauswertung in der Gamestate-, ADSM-, als auch der Animations-Schicht genutzt werden können, wodurch die Verantwortlichkeiten durch den Animationsclip nicht automatisch klar sind, eine Folge der Querkopplung.

Das SoC verlangt nicht zwingend, dass alles für die Berechnung der Bewegung in einem System sein muss, sondern betont, dass Systeme sich nicht in ihren Verantwortlichkeiten überlappen sollten (vgl. Anguelov 2019, S.106). In Kapitel 3 wurde gezeigt, dass beispielsweise im Falle einer ADSM-Schicht die Verantwortungen so getrennt werden können, dass die Gamestate-Schicht Veränderungen über Parameter auslösen kann, und die ADSM-Schicht die Clip- und Blendwahl anhand dieser vornimmt, während das Root-Delta in der Animations-Schicht berechnet wird. Jeder entscheidende Aspekt in einem System muss eine eindeutige Lage der Verantwortung haben, damit Änderungen (z. B. an Code/ADSM-Struktur/Parameter) nicht ungewollt das Gameplay invalidiert und die Wartungsarbeit erschwert (vgl. Kap. 3.2–3.4) (vgl. Anguelov 2019, S.104).

Kriterium 2:

Die Skalierung des Moveset ist beherrschbar.

Root Motion ist geeigneter, wenn die Anzahl der Aktionen und die ADSM-Zustände/Übergänge strukturiert oder begrenzt bleibt. Anguelov beschreibt, sobald jede neue Aktion in der Praxis (1) neue Clips, (2) Übergänge zwischen den Aktionen und (3) zusätzliche Parameter-/Driver-Logik nach sich zieht, können Wartungs- und Debug-Aufwand stark steigen, sodass sie im späteren Verlauf der Entwicklung kaum noch zu rechtfertigen sind (vgl. Kap. 3.2–3.4) (vgl. Anguelov 2019, S.105).

Kriterium 3:

Die Clipabdeckung ist ausreichend für die intendierten Aktionsvarianten.

Root Motion ist geeignet, wenn der Clipbestand (vorhandene Assets) das Aktionsset und die relevanten Varianten in Trajektorie, Timing und ggf. einer geeigneten Pose-Ausführung abdeckt oder zuverlässig kombinierbar macht. Bei mangelnder Abdeckung wird die Querkopplung zum Problem der Intentions-Unterstützung, eine gewünschte Bewegungsintention kann bei einer clipbasierten Bewegungssteuerungen nur bei vorhandenen Clips erfüllt werden (vgl. Kap. 2.3, Kap. 4.5) (vgl. Pichlmair und Johansen 2020, S.1).

Kriterium 4:

Commitment von Aktionen ist durch vorgegebene Sequenzen gewollt.

Das Commitment einer Aktion kann besser geplant werden, wenn die Bewegungswirkung einer Aktion voraussagbar ist. Bei Root Motion ergibt sich die Bewegung nicht aus einem separaten dynamisch berechneten Bewegungsmodell, sondern durch das Abspielen eines Animationsclips, einer Sequenz mit Phasen (Aufholen, Impakt, Erholen), die in ihrem Design zugunsten einer gewünschten Wirkungen gestaltet werden können (vgl. Lindgren et. al. 2025, S.30).

Bei identischer Sequenz und Übergangspfad ergibt sich eine

reproduzierbare Bewegungstrajektorie, sprich sie ist voraussagbar. Dieses Commitment kann jedoch auch als verringerte Kontrolle wahrgenommen werden (vgl. Kap. 4.5). Konstante Korrekturen an der Trajektorie (z. B. Steering) sind eine Möglichkeit die Kontrolle zu erhöhen, jedoch ist diese Präzisionskontrolle bei Root Motion nicht automatisch gegeben (vgl. Kap. 2.2). Deswegen ist Root Motion besonders passend, wenn Commitment an einer vorgegebenen Sequenz eines Animationsclips Teil der gewünschten Bewegungswirkung ist, andernfalls steigt das Risiko eines wahrgenommenen Kontrollverlusts.

Kriterium 5:

Responsivität-Budget passt zum Aktionsdesign.

Eine hohe Responsivität kann die Kontrolle der Bewegungssteuerung direkt verbessern (vgl. Kap. 4.2). Früh sichtbares Feedback und klar definierte Aktionszeitfenster können jedoch nicht bei jedem Animationsclip oder Blendpfad gewährleistet werden (vgl. Kap. 4.3). Deswegen zeigt sich Root Motion geeignet, wenn die Zeit bis zur Bewegungswirkung nicht unnötig durch eine ungeeignete Animationssequenz oder lange Übergänge verlängert wird. Wenn aber doch die Responsivität zugunsten der Bewegungsplausibilität nach hinten gestellt wird, dann muss dies bewusst einkalkuliert sein. Denn empirisch ist eine hohe Responsivität besonders wichtig, wenn Aufgaben anspruchsvoll sind, was eher wahrscheinlich in einem Third-Person Action Setting ist (vgl. Jörg et. al. 2012, S.1).

Kriterium 6:

Bewegungsplausibilität ist ein gewolltes Ziel.

Bei Root Motion gibt es zugleich mehrere Mechanismen, die eine erhöhte Bewegungsplausibilität begünstigen. Durch den Nutzen einer ADSM-ähnliche Systemschicht, ist die Verantwortung über die Handhabung der Clips an einem Ort gebündelt, dort kann die Bewegung, aufgrund der Querkopplung zusammen mit der Pose, auf der Ebene der Clipinhalte bearbeitet und angewendet werden (vgl. Kap. 3.5 & 4.5). Und aufgrund der Zeitkopplung werden diese aus demselben Zeitpunkt ausgewertet, wodurch sie synchron abgespielt werden können (vgl. Kap. 4.5).

Außerdem können Verzögerungen, die explizit durch das Animationssystem entstehen, um eine realistische Bewegungs- und Pose-Ausführung zu erzeugen, die Bewegungsplausibilität steigern, verzögern jedoch aber auch die Bewegungswirkung (vgl. Normoyle und Jörg 2014a, S.2).

Jedoch dominiert die Bewegungsplausibilität nicht automatisch gegenüber der Kontrolle durch eine hohe Responsivität (vgl. Kap. 4.3) (Normoyle und Jörg 2014a, S.1). Daher ist Root Motion besonders geeignet, wenn Bewegungsplausibilität als Ziel klar priorisiert oder der Responsivität-Preis bewusst begrenzt/kompensiert werden kann.

Kriterium 7:

Die Wirksamkeit einer Aktion ist so konsistent, dass sie nicht erschwert zu lernen ist.

Aufgrund der Blendkopplung kann es dazu führen, dass die Wirksamkeit einer Bewegung je nach Startzustand und Blendpfad variieren kann (vgl. Kap. 4.5). Das kann geschehen, wenn Blendpfade nicht für eine konsistente Bewegungswirkung, sondern z. B. zugunsten eines realistischen Posenübergangs gestaltet sind. Diese Variabilität kann die Vorhersagbarkeit beeinflussen. Empirisch ist eine variierende Verzögerung, die die Lernbarkeit der Bewegungssteuerung beeinträchtigt, besonders schädlich für das Kontrollgefühl und die Performance (vgl. Kap. 4.4) (vgl. Normoyle et. al. 2014b, S.1). Root Motion ist deswegen dann geeignet, wenn Übergangsdauern, Blendpfade und Wirksamkeitsfenster so gestaltet werden können, dass die effektive Aktionsantwort konsistent ist und erlernbar bleibt. Andernfalls leidet das Kontrollgefühl, was besonders wichtig in Third-Person Action Games ist (vgl. Lindgren et. al. 2025, S.14).

5.3 Hinweise zu den Kriterien

Die hier sieben aufgezählten Kriterien wurden aus den Befunden aus Kapitel 3 und 4 abgeleitet. Diese Kriterien decken hauptsächlich den Nutzen von Root Motion mit Betrachtung auf System aus der SoC Perspektive und den Einfluss auf das Spielerlebnis durch die Qualitätsmerkmale der Kontrolle der Bewegungssteuerung ab. Die formulierten Kriterien sind ungetestet und können womöglich Aspekte, die in dieser Arbeit nicht betrachtet worden sind, vernachlässigen. Daher sollten diese nicht als absolut betrachtet werden. Einzelne Kriterien können womöglich in präzise formulierte Kriterien aufgeteilt oder zusammengetragen werden, oder ganz durch neue Erkenntnisse erweitert oder ersetzt werden.

Nichtsdestotrotz sollten die betrachtenden Aspekte der Arbeit genug Evidenz erbracht haben, um diese Kriterien als erste Iteration von Voraussetzungen für den Nutzen von Root Motion zu liefern. Sind jedoch diese Bedingungen der Entwicklung der Bewegungssteuerung nicht erfüllbar, beispielsweise durch den Mangel an passenden Animationsclips, den Bedarf an kontinuierlicher Korrekturfähigkeit der Bewegungstrajektorie oder bei engem Responsivitäts- und Konsistenzbudget, ist es folgerichtig, eine alternative Bewegungsautorität mit womöglich stärkeren gamestate-autorisierten Ansatz vorzuziehen.

6. Abschluss und Ausblick

6.1 Rückblick auf die Ziele der Arbeit

Ziel dieser Arbeit war es, **I)** Root Motion anhand seiner Eigenschaften zu definieren, dafür wurde die Technik konzeptionell formal ausgelegt. Durch die daraus operationalisierten Kopplungsdimensionen (Quell-, Zeit- und Blendkopplung) konnte eine Zentrale Eigenschaft von Root Motion, die strukturelle Bindung von Bewegung und Pose, definiert werden.

Daraufhin wurde **II)** die Auswirkung von Root Motion auf das System und Spielerlebnis betrachtet. Dafür wurde zuerst die Auswirkung der Quellkopplung auf das System anhand des Separation of Concerns Prinzips analysiert. Die Analyse hat gezeigt, dass Root Motion in erster Linie als Entscheidung über Bewegungsautorität zu verstehen ist, die durch das Root-Delta und die Clip- und Blendwahl einer ADSM nicht ausschließlich in der Gamestate-Schicht liegt. Anschließend wurden die Auswirkungen auf das Spielerlebnis analysiert. Es stellte sich raus, dass Root Motion Einfluss auf das Kontrollgefühl der Bewegungssteuerung haben kann, indem es besonders die Bewegungsplausibilität begünstigt, während Responsivität und dessen Konsistenz Clip- und Blendpfad-abhängig variieren kann.

Damit konnte die Annahme in der Einleitung überprüft werden, dass die Bewegungssteuerung bei Root Motion tatsächlich die Bewegungsautorität über die Systeme vermischt und in einem Spannungsfeld zwischen Kontrollgefühl und einer realistischen Bewegungsausführung steht.

Zum Schluss wurden aus der vorherigen Analyse Kriterien abgeleitet, die **III)** die Forschungsfrage zu der Eignung von Root Motion in One-Shot Aktionen beantworteten. Root Motion ergibt sich als plausibel, wenn (K1) die Bewegungsautorität zwischen der Gamestate- und Animations-Schicht explizit und stabil definiert ist, (K2) die Skalierung des Movesets, durch z. B. Übergänge und Parameter beherrschbar, bleibt und (K3) der Clipbestand die relevanten Varianten zuverlässig abdeckt. Dann ist Root Motion vor

allem geeignet, wenn (K4) Commitment als sequenzierte Aktion gewollt ist, (K5) die Wirksamkeit nicht durch Übergänge/Sequenzphasen unnötig verzögert wird, (K6) eine möglicher verminderte Responsivität zugunsten von der Bewegungsplausibilität bewusst akzeptiert oder kompensiert wird, und (K7) die Wirksamkeit der Bewegung über zustandsabhängige Blendpfade konsistent und erlernbar gestaltet werden kann.

6.2 Beitrag der Arbeit

Diese Arbeit versucht an der Lösung, von einem Mangel an einer Standarddefinition von Root Motion, beizutragen, die bisher hauptsächlich nur über die Dokumentationen von Game-Engines zu ihren eigenen Features und einem möglichen Allgemeinverständnis abgedeckt ist. Der Beitrag besteht aus einer formalen Operationalisierung der Kopplung bei Root Motion, anhand dessen man den Nutzen der Technik in der Produktion von Videospielen besser planen kann. Dazu steuern auch die Befunde aus Kapitel 3 und 4 bei, die konkreter darauf eingehen, wie sich die Auswirkungen von Root Motion in einem System oder dem Spielerlebnis aufzeigen. Zum Schluss wurde versucht, allgemeine Kriterien für den Nutzen von Root Motion zu erstellen, die in dieser Arbeit den Nutzen für One-Shot Aktionen in Third-Person Action Games abdecken.

6.3 Grenzen der Analyse

Die Arbeit ist eine literaturgestützte, konzeptionelle Analyse und erhebt keine eigenen empirischen Daten. Die Übertragung der in Kapitel 3 und 4 berichteten Befunde auf Root Motion Aktionen erfolgt daher teilweise als Hypothese. Zudem sind die herangezogenen empirischen Studien in spezifischen Versuchssettings entstanden, in denen Steuerungsschemata und Movesets begrenzt waren mit möglichem Fokus auf unterschiedliche Subgenres von Third-Person Action Games und ohne Einschränkung auf One-Shot Aktionen. Dies ersetzt keine direkte Untersuchung in einem Root-Motion-fokussierten Setting von Charakteraktionen in Third-Person Action Games. Schließlich wurde bewusst Engine-spezifische Implementationslogik abstrahiert, die Ergebnisse beziehen sich entsprechend auf die systemische Logik von Root Motion und nicht auf konkrete Engine-Features.

6.4 Ausblick

Aus den Ergebnissen ergeben sich drei naheliegende nächste Schritte, zum einen sollte die Transferhypothese der variierenden Verzögerungen durch Blendpfade in einem prototypischen Third-Person-Combat-Setting empirisch geprüft werden, etwa über kontrollierte Variationen der Übergangsdauer und einer Messung der Lernkurve, Fehlerraten und subjektive Kontrolle.

Zum anderen bietet sich vor allem eine empirische Prüfung der abgeleiteten Kriterien an. Das könnte überprüft werden, indem ein oder mehrere Prototypen einer Bewegungssteuerung für unterschiedliche Zielsetzungen gebaut werden. Es können konkrete Ziele wie Bewegungsplausibilität, Responsivität oder die System-Wartbarkeit verfolgt werden, oder Ziele, wie die Minimierung des Arbeitsaufwand an einer Bewegungssteuerung im Vergleich zu anderen Bewegungstechniken im Kontext von One-Shot Aktionen. Anhand der Ergebnisse können die Kriterien angepasst oder verfestigt werden.

Zuletzt könnte man die Analyse auch erweitern, indem man den Fokus von One-Shot Aktionen auf Lokomotion erweitert oder den Nutzen von Root Motion bei anderen Genres wie First-Person-Shooter analysiert.

Quellenverzeichnis

Anguelov, Bobby (2019): Separation of Concerns. In: Game AI Pro 360: Guide to Architecture. CRC Press. p. 253–268.

Dijkstra, Edsger W. (1982): On the role of scientific thought. In: Selected Writings on Computing: A Personal Perspective.

Dori_Yoon (2023): Issues with movement/rotation changes due to blending in root motion montages. In: Epic Developer Community Forums. Online unter: <https://forums.unrealengine.com/t/issues-with-movement-rotation-changes-due-to-blending-in-root-motion-montages/1351928>. (Letzter Zugriff am 02.02.2026)

Epic Games (2025a): Root Motion in Unreal Engine. In: Epic Games Developer. Epic Games. Online unter: <https://dev.epicgames.com/documentation/en-us/unreal-engine/root-motion-in-unreal-engine>. (Letzter Zugriff am 26.02.2026)

Epic Games (2025b): Understanding Networked Movement in the Character Movement Component for Unreal Engine. In: Epic Games Developer. Online unter: <https://dev.epicgames.com/documentation/en-us/unreal-engine/understanding-networked-movement-in-the-character-movement-component-for-unreal-engine>. (Letzter Zugriff am 26.02.2026)

Jörg, Sophie/Normoyle, Aline/Safonova, Alla (2012): How responsiveness affects players' perception in digital games. In: Proceedings of the ACM Symposium on Applied Perception. New York, NY, USA: ACM. p. 33–38. Online unter: <https://doi.org/10.1145/2338676.2338683>. (Letzter Zugriff am 13.01.2026)

Lindgren, Anton/Barbosa, João/Dlugoš, Patrik (2025): Striking Balance: Merging Animation and Control Elements for a Better Feel of Attacks: An Exploration into the Improvement of Visual Feedback and Input Responsiveness in a Hack-&-Slash Game. In: DIVA. Online unter: <https://uu.diva-portal.org/smash/record.jsf?pid=diva2%3A1981485&dsid=-2948>. (Letzter Zugriff am 14.01.2026)

Normoyle, Aline/Jörg, Sophie (2014a): Trade-offs between responsiveness and naturalness for player characters. In: Proceedings of the Seventh International Conference on Motion in Games. New York, NY, USA: ACM. p. 61–70. Online unter:

<https://doi.org/10.1145/2668064.2668087>. (Letzter Zugriff am 13.01.2026)

Normoyle, Aline/Guerrero, Gina/Jörg, Sophie (2014b): Player perception of delays and jitter in character responsiveness. In: Proceedings of the ACM Symposium on Applied Perception. New York, NY, USA: ACM. p. 117–124. Online unter: <https://doi.org/10.1145/2628257.2628263>. (Letzter Zugriff am 13.01.2026)

Pichlmair, Martin/Johansen, Mads (2020): Designing Game Feel. A Survey. In: arXiv.org. Online unter: <https://arxiv.org/abs/2011.09201>. (Letzter Zugriff am 13.01.2026)

Open 3D Engine (2025): O3DE: Data Driven Root Motion. In: Open 3D Engine. Online unter: <https://docs.o3de.org/docs/learning-guide/tutorials/animation/data-driven-root-motion/>. (Letzter Zugriff am 26.02.2026)

Pusche (2020): Reasons to not use Root Motion. In: Game Development Stack Exchange. Online unter: <https://gamedev.stackexchange.com/questions/174362/reasons-to-not-use-root-motion>. (Letzter Zugriff am 02.02.2026)

Unigine (2026): Unigine Developer. In: Unigine Developer. Online unter: https://developer.unigine.com/ch/docs/future/content/animations/root_motion/. (Letzter Zugriff am 27.02.2026)

Unity Technologies (2025): Unity. In: Manual: How Root Motion works. Unity Technologies. Online unter: <https://docs.unity3d.com/6000.3/Documentation/Manual/RootMotion.html>. (Letzter Zugriff am 26.02.2026)

Unity Technologies (2026): Unity. In: Manual: Skinned Mesh Renderer component reference. Online unter: <https://docs.unity3d.com/6000.3/Documentation/Manual/class-SkinnedMeshRenderer.html>. (Letzter Zugriff am 26.02.2026)

Vaei (2016): Separate Animation Blending Time for Root Motion from the actual animation. In: Epic Developer Community Forums. Online unter: <https://forums.unrealengine.com/t/separate-animation-blending-time-for-root-motion-from-the-actual-animation/68051>. (Letzter Zugriff am 02.02.2026)

Eidesstattliche Versicherung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe und dass die Arbeit in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt wurde. Alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, sind als solche gekennzeichnet. In der Anlage „Dokumentation der Verwendung von KI-Werkzeugen“ habe ich dargelegt, welche KI-Werkzeuge ich genutzt habe, für welchen Zweck ich diese verwendet habe und auf welche Weise die Nutzung stattfand.

27.03.2026

Datum


Unterschrift

Dokumentation der Verwendung von KI-Werkzeugen

Für die Bachelorarbeit *“Kriterien für den Nutzen von Root Motion; Eine system- und erlebnisbezogene Analyse”* wurde bei der Recherche ChatGPT, NotebookLM und Perplexity genutzt. Die Ergebnisse der Recherche von ChatGPT und NotebookLM sind teilweise in der Bachelorarbeit genutzt worden.

Während des Schreibens der Rohfassung wurde ChatGPT 5.2 genutzt, dessen Texte verteilt in der Arbeit als Gerüst genutzt wurden. Jedoch sind diese Textabschnitte durch mehrere Iterationen eigener Anpassungen verändert und korrigiert worden.

Folgende Tabelle nennt den konkreten Nutzen der KI-Tools

Verwendungszweck	KI-Tool	Verwendungsort
Unterstützung bei der Gliederung	ChatGPT 5.2	Die einzelnen Abschnitte in Kapitel 2, 3 und 4 wurden vorgeschlagen, wurden dann aber von mir zum Teil umstrukturiert.
Unterstützung bei der korrekten inhaltlichen Wiedergaben und Formulierung von englischen Texten	ChatGPT 5.2 NotebookLM	Die Quellen die in den Kapiteln 3.1, 3.2, 3.4, 4.1, 4.2, 4.3, 4.4 aufgegriffen und inhaltlich wiedergegeben wurden, wurden in der Rohfassung von KI-Tools erstellt und im späteren Verlauf nach eigenen Anpassungen erneut von ChatGPT überprüft und korrigiert.
Hilfe bei der Formulierung von Absätzen	ChatGPT 5.2	Das trifft zu auf: - das Abstrakt - die erste Hälfte der Einleitung - Kapitel 2 (die Herleitung der Formeln ausgeschlossen) - Kapitel 3 und 4
Schreiben der Formeln	ChatGPT 5.2	ChatGPT hat geholfen, LaTeX Formeln für Kapitel 2 zu kreieren
Formulieren der Kriterien	ChatGPT 5.2	ChatGPT hat eine Liste an Vorschlägen für mögliche Kriterien gemacht.
Formale und Sprachliche Korrekturen	ChatGPT 5.2	Das trifft auf die ganze Arbeit zu.